

LAB MANUAL

WEB ENGINEERING II



DEPARTMENT OF SOFTWARE ENGINEERING
FACULTY OF ENGINEERING & COMPUTING
NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

TABLE OF CONTENTS

Preface.....	3
Tools/ Technologies.....	3
Lab Outline	4
LAB 1: PHP Pagination	6
LAB 2: Working with jQuery + Ajax	10
LAB 3: Node JS Installation	15
LAB 4: Introduction to React JS and JSX	18
LAB 5: Working with React JS Components	21
LAB 6: Working with States in React JS.....	24
LAB 7: React UI styling with States.....	27
LAB 8: Working with React API fetch().....	32
LAB 9: Midterm Week	36
LAB 10: Working with React Routers.....	37
LAB 11: Intro to MongoDB CRUD operations	40
LAB 12: Intro to MERN stack App backend.....	48
LAB 13: Working with MongoDB Schemas in MERN backend.....	51
LAB 14: Understanding Backend Routers with MERN & API testing tools.....	53
LAB 15: Project Evaluation.....	56
LAB 16: Final Term Exam	57
Open Ended Lab (OEL)	58

Preface

This lab manual has been prepared to facilitate the students of software engineering in studying and implementing Advanced Web Engineering concepts. The students will learn React JS development related to Front End development as well as MERN back-end development. After completing this course, students will be able to implement MERN stack applications practically.

Tools/ Technologies

- HTML, CSS
- JavaScript, jQuery
- React JS
- MongoDB
- Express JS
- Node JS
- Thunderclient/Postman
- VS Code

Lab Outline

I. Course Details

Credit Hours	3 (2+1)
Pre-requisites	Web Engineering
Recommended Textbook(s)	Web Development with JQuery, Richar York, Wiley, 2015
Recommended Reference (Books/Websites/Articles)	1. PHP Web Services, O'Reilly Media, Inc, (2nd edition) 2013 2. The Road to Learn React: Your Journey to Master Plain Yet Pragmatic React. Js, Robin Wieruch, (2020 edition), 2020 3. https://react.dev/blog/2023/03/16/introducing-react-dev 4. https://www.w3schools.com/nodejs/

II. Lab Learning Outcomes (CLO)

CLOs	Description	Domain	Taxonomy level	PLOs	Assessment Artifacts
CLO-1	Demonstrate the task by interpreting, summarizing, and explaining key concepts/ principles / relationships within the domain, enabling them to apply acquired knowledge effectively in diverse contexts.	Cognitive	2	2	Lab Task, Lab Report, Final Exam, Semester Project
CLO-2	Practice theoretical concepts for conducting the experiments / projects of varying complexities to achieve desired outcomes.	Psychomotor	3	4	Lab Task, Lab Report, Final Exam, Semester Project
CLO-3	Practice underlying theoretical concepts through modern tool usage to achieve desired outcomes.	Psychomotor	3	5	Lab Task, Final Exam, Semester Project

CLO-4	Contribute actively and responsibly individually or in teams during laboratory activities, demonstrating a positive attitude and willingness to participate.	Affective	2	6	Lab Task, Final Exam, Semester Project
--------------	---	-----------	---	---	--

III. Lab Assessment

Evaluation Methods	Weight (%)
Internal Evaluation (Lab Report/Lab Task/Lab Project)	60
Open Ended Lab	15
Final Term Exam	25
Total	100

IV. Weekly Breakdown (All CLOs shall be assessed in every lab)

Week No.	Topics
1	Pagination with PHP
2	Working with jQuery + Ajax
3	Tools Installation (Node JS)
4	Introduction to React JS and JSX
5	Working with States in React JS
6	React UI styling with States
7	React UI Styling using States
8	Working with React API fetch()
9	Midterm Exams
10	React routers
11	Introduction to MongoDB CRUD operations
12	Introduction to MERN stack App backend
13	Working with MongoDB Schemas in MERN backend
14	Understanding Backend Routers with MERN and API testing tools
15	Project Evaluation
16	Final Term Exam

*Open ended lab after week 8

LAB 1: PHP Pagination

Objectives

1. Revise Database concepts in MySQL
2. Apply pagination on PHP Database

Theoretical Description

Creating Pagination with PHP and MySQL

PHP Pagination

PHP is mostly used to store and display data from a database. Pagination can be done with ajax, but here this is done with non-ajax. In this tutorial, we will learn the pagination in PHP with MySQL. Let's take a brief review about pagination with an example

Pagination is a common task for PHP developers. MySQL helps the developer to create pagination by using LIMIT clause, which takes two arguments. The first argument as OFFSET and the second argument is number of records that will return from database.

Simple steps to create pagination -

- Create a database and table. Provide a list of 100 records into the table.
- Connect with the MySQL database.
- Create the pagination link to split the data on multiple pages and add them to bottom of the table.
- Fetch data from the database and display it to the multiple pages.

Follow the following steps:

Steps:

Create index.php to display paginated data.

Database creation

First of all, create a database named by **taskManager** as we created and a table inside it named **tasks**. Create attribute by name **taskID** and **taskName**, and provide the data in the table.

Insert 100 records in the database.

Create a Connection File

Create a connection File databaseConnect.php

```
<?php
    $servername = "localhost";
    $username = "root";
    $password = "";
    $database = "TaskManager";
    // Create connection
    $conn = new mysqli($servername, $username, $password, $database);
    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    // Connected successfully

?>
```

Create index.php

Create index file and include the dbconnection file:

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
```

```
<?php

include('databaseConnect.php');

?>
```

Get the current page number

```
if (!isset($_GET['page'])) {
    $page = 1;
} else {
    $page = $_GET['page'];
}
```

Formula for pagination

For pagination, you need to set the limit of number of records to be displayed per page. Here, we set the limit for result per page to 10, so it will display on each page

```
$results_per_page = 10;
$page_first_result = ($page-1) * $results_per_page;
```

Get total number of pages

```
// Get total number of pages
```

```
$query = "select * from tasks";
$result = mysqli_query($conn, $query);
$number_of_result = mysqli_num_rows($result);
```

```
//determine the total number of pages available
$number_of_page = ceil ($number_of_result / $results_per_page);
```

Retrieve Data on web page

```
<center>
<table height = "50" cellpadding =10px width=80% border = "1" >
<tr> <th>taskID</th> <th>Task Name</th> </tr>
<?php
$query = "SELECT *FROM tasks LIMIT " . $page_first_result . ' ' . $results_per_page;
$result = mysqli_query($conn, $query);
```

```
while ($row = mysqli_fetch_array($result))
{
    echo "<tr style = 'background-color : #FFFFE0'>
    <td>",$row['taskID'], "</td><td>",$row['taskName'], "
    <td colspan='\"8\"' style='display:none;' ",$row['taskName'],""></td>
    </tr>";
}
?>
</table>
```

Display the link of the pages in URL

```
//Display the link of the pages in URL
```

```
for($page = 1; $page<= $number_of_page; $page++) {
    echo '<a href = "index.php?page=' . $page . '">' . $page . ' </a> &nbsp; &nbsp; &nbsp;'; }

?>

</center>
```

Final Output

taskID	Task Name
11	Important Task
12	Important Task
13	Important Task
14	Important Task
15	Important Task
16	Important Task
17	Important Task
18	Important Task
19	Important Task
20	Important Task

1 2 3 4 5 6 7 8 9 10 11

Lab Tasks

Follow the above tutorial.

1. Create a record of 50 tasks and display the record in 10 pages. Provide the relevant code and output.
2. Style the page links [1, 2 ,3 ,4 ...] with different hover effects.

LAB 2: Working with jQuery + Ajax

Objectives

- Introduce jQuery concepts
- Introduce Ajax concepts
- Apply pagination on PHP Database

Theoretical Description

jQuery: jQuery load() Method

The jQuery load() method is a simple, but powerful AJAX method.

The load() method loads data from a server and puts the returned data into the selected element.

Syntax

Syntax:

`$(selector).load(URL,data,callback);`

1. Create a basic web page

Index.php

index.php

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Jquery AJAX, PHP and MySQL </title>
```

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script></script>
```

```
<script src="https://code.jquery.com/jquery-3.2.1.min.js"></script>
```

```
</head>
```

```
<body>
```

```
<div style="width:85%;">
```

```
<h2>Pagination with Jquery AJAX, MySQL and PHP </h2>
```

```
</div>
```

```
</body>
```

```
</html>
```

2. Add the following in the body section

```
<div id="content_container"></div>
```

Add the following script to the body section

```
$(document).ready(function(){
```

```
$("#content_container").load("ajaxpagination.php?page=1");
```

```
});
```

3. Create file ajaxpagination.php

```
<?php
```

```
include("databaseConnect.php");
```

```
?>
```

4. Create file databaseConnect

```
<?php
```

```
$servername = "localhost";
```

```
$username = "root";
```

```
$password = "";
```

```
$database = "TaskManager";
```

```
// Create connection
```

```
$conn = new mysqli($servername, $username, $password, $database);
```

```
// Check connection
```

```
if ($conn->connect_error) {
```

```
die("Connection failed: " . $conn->connect_error);
```

```
}
```

```
// Connected successfully
```

```
?>
```

```
Ajaxpagination.php
```

```
$per_page = 20;
```

```
if(isset($_GET['page'])) {
```

```
$page=$_GET['page'];
```

```
}
```

```
$start = ($page-1)*$per_page;
```

```
$sql_query = "SELECT * FROM tasks ORDER BY taskID LIMIT $start,$per_page ";
```

```
$resultset = mysqli_query($conn, $sql_query);
```

```
Agaxpagination.php
```

5. Add following code to display data coming from database:

```
<table width="100%">
```

```
<thead>
```

```
<tr>
```

```
<th> Id</th>
```

```
<th>Task Name</th>
```

```
</tr>
```

```
</thead>
```

```
<?php
```

```
while($rows = mysqli_fetch_array($resultset)){ ?>
<tr bgcolor="#DDEBF5">
<td> <?php echo $rows['taskID']; ?></td>
<td><?php echo $rows['taskName']; ?></td>
</tr>
```

6. Index.php

Add this above content_content_container div

```
<div id="loading"></div>
Include the following
In document.ready function script above content_content_container div
function hideLoading() {
$("#loading").fadeOut('slow');
};
```

Update load function:

```
$("#content_content_container").load("ajaxpagination.php?page=1", hideLoading());
```

7. Load in browser

Ajaxpagination.php

Now we need to specify per page records:

For that, we'll add the following before for extracting data on page:

```
$per_page = 20;
if(isset($_GET['page'])) {
    $page=$_GET['page'];
}
$start = ($page-1)*$per_page;
```

```
$sql_query = "SELECT * FROM tasks ORDER BY taskID LIMIT $start,$per_page ";
```

Load in browser

Pagination links

Now we add links to other pages. For this, open index.php

Add following logic in first php tag

```
$per_page = 20;
$sql = "SELECT * FROM tasks";
$result = mysqli_query($conn, $sql);
$count = mysqli_num_rows($result);
$pages = ceil($count/$per_page)
Pagination class
Add a pagination class in content_content_container
<div class="pagination" style="padding:10px;">
<ul id="paginate">
```

```
<?php
for($i=1; $i<=$pages; $i++) {
echo '<li id="'. $i. "'>'. $i. '</li>';
}
?>
</ul>
</div>
```

Add style to index.php

```
<style>
.pagination{
float:left;
width:100%;
}
.botoom_link{
float:left;
width:100%;
}
.content{
float:left;
width:100%;}
ul {
list-style: none;
float:left;
margin:0;
padding: 0;
}
li{
list-style: none;
float: left;
margin-right: 16px;
padding:5px;
border:solid 1px #dddddd;
color:#0063DC;
}
li:hover{
color:#FF0084;
cursor: pointer;
}

```

```
</style>
```

Add styles in script

```
$("#paginate li:first").css({'color' : '#FF0084'}).css({'border' : 'none'});
$("#content_container").load("ajaxpagination.php?page=1", hideLoading());
$("#paginate li").click(function(){
$("#paginate li").css({'border' : 'solid #dddddd 1px'}).css({'color' : '#0063DC'});
```

```
$(this).css({'color' : '#FF0084'}).css({'border' : 'none'});  
var page_num = this.id;  
$("#content_container").load("ajaxpagination.php?page=" + page_num, hideLoading());
```

Lab Tasks:

1. Follow the above tutorial.
2. Explore jQuery methods from w3schools tutorial:
<https://www.w3schools.com/jquERy/default.asp>
3. Implement Infinite Scroll by utilizing jQuery. scroll method.

LAB 3: Node JS Installation

Objectives

1. Node JS installation
2. Working with Node JS. Sending an email with Node JS

Theoretical Description

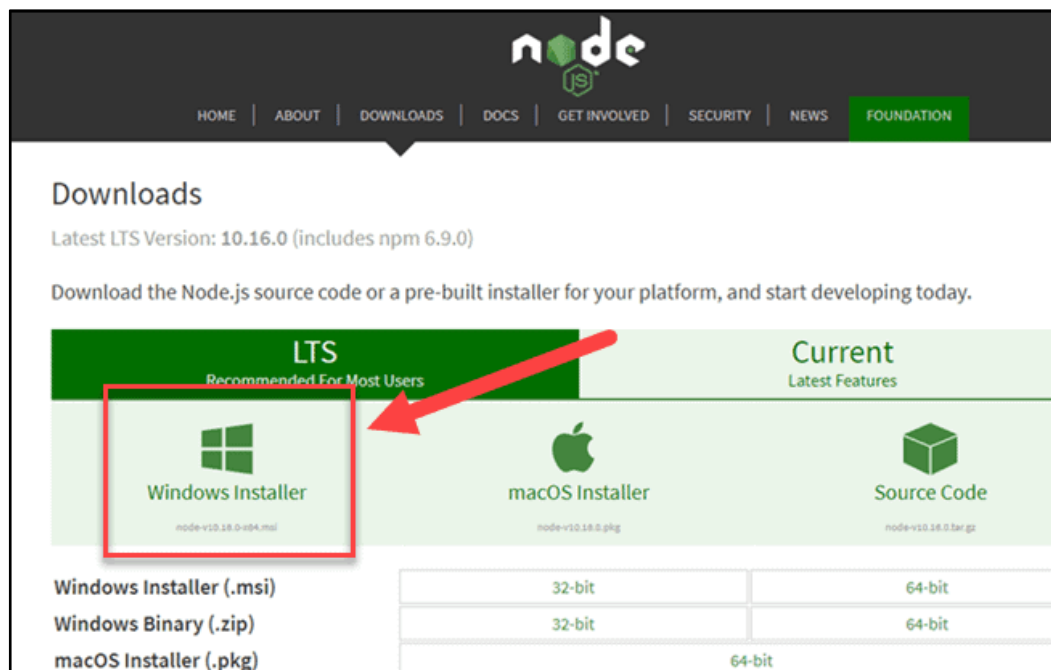
Node.js is a run-time environment which includes everything you need to execute a program written in JavaScript. It's used for running scripts on the server to render content before it is delivered to a web browser. NPM stands for Node Package Manager, which is an application and repository for developing and sharing JavaScript code.

Prerequisites

A user account with administrator privileges (or the ability to download and install software)
 Access to the Windows command line (search > cmd > right-click > run as administrator) OR
 Windows PowerShell (Search > Powershell > right-click > run as administrator)
 How to Install Node.js and NPM on Windows

Step 1: Download Node.js Installer

In a web browser, navigate to <https://nodejs.org/en/download/>. Click the **Windows Installer** button to download the latest default version. At the time this article was written, version 10.16.0-x64 was the latest version. The Node.js installer includes the NPM package manager.



Note: There are other versions available. If you have an older system, you may need the 32-bit version. You can also use the top link to switch from the stable LTS version to the current version. If you are new to Node.js or don't need a specific version, choose LTS.

Step 2: Install Node.js and NPM from Browser

1. Once the installer finishes downloading, launch it. Open the **downloads** link in your browser and click the file. Or, browse to the location where you have saved the file and double-click it to launch.
2. The system will ask if you want to run the software – click **Run**.
3. You will be welcomed to the Node.js Setup Wizard – click **Next**.
4. On the next screen, review the license agreement. Click **Next** if you agree to the terms and install the software.
5. The installer will prompt you for the installation location. Leave the default location, unless you have a specific need to install it somewhere else – then click **Next**.
6. The wizard will let you select components to include or remove from the installation. Again, unless you have a specific need, accept the defaults by clicking **Next**.
7. Finally, click the **Install** button to run the installer. When it finishes, click **Finish**.

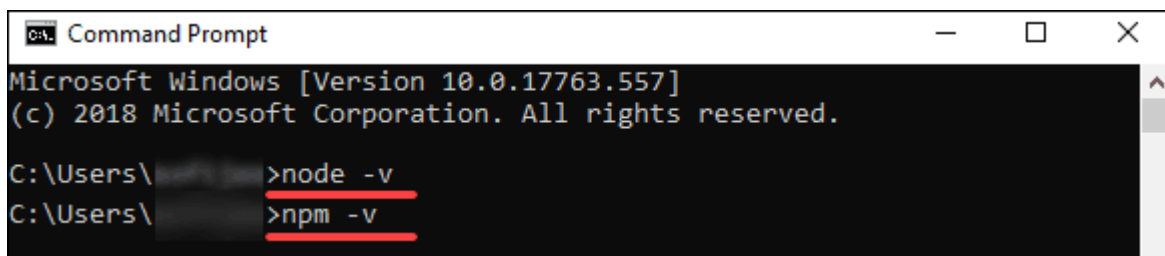
Step 3: Verify Installation

Open a command prompt (or PowerShell), and enter the following:

```
node -v
```

The system should display the Node.js version installed on your system. You can do the same for NPM:

```
npm -v
```



The screenshot shows a Windows Command Prompt window titled 'Command Prompt'. The window displays the following text: 'Microsoft Windows [Version 10.0.17763.557] (c) 2018 Microsoft Corporation. All rights reserved.' Below this, the user is at the 'C:\Users\' prompt. Two commands are entered and executed: '>node -v' and '>npm -v'. Both commands are highlighted with red underlines. The output of these commands is not visible in the screenshot.

Basic Node.js Usage

Node.js is a framework, which means that it doesn't work as a normal application. Instead, it interprets commands that you write. To test your new Node.js installation, create a **Hello World** script.

1. Start by launching a text editor of your choice.
2. Next, copy and paste the following into the text editor you've just opened:

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

3. Save the file, then exit. Open the PowerShell, and enter the following:

```
node \users\<your_username>\myprogram.js
```


It will look like nothing has happened. In reality, your script is running in the background. You may see a Windows Defender notice about allowing traffic – for now, click **Allow**.

4. Next, open a web browser, and enter the following into the address bar:

`http://localhost:8080`

In the very upper-left corner, you should see the text ***Hello World!***

Right now, your computer is acting like a server. Any other computer that tries to access your system on port 8080 will see the *Hello World* notice.

To turn off the program, switch back to PowerShell and press **Ctrl+C**. The system will switch back to a command prompt. You can close this window whenever you are ready.

Lab Tasks

1. Install Node.js on your machine by following above manual
2. Verify installation and prepare lab manual
3. Create an app to send Email from Node.js. Follow the tutorial on:

https://www.w3schools.com/nodejs/nodejs_email.asp

For reference:

<https://blog.logrocket.com/send-emails-nodejs-nodemailer/>

LAB 4: Introduction to React JS and JSX

Objectives

1. React JS installation
2. Creating first React app
3. Understanding JSX in React

Theoretical Description

- Create React App

`npx create-react-app myapp`

- This takes time and creates a full-fledged app in a folder myapp
- The workaround is:
`npx create – react – app`
- This will create all the files in the explorer view
- Type the following:

`npx create – react – app MyApp`

- Start the server

Once app is created, type in **`npm start`**

The React app should be loaded in your browser

Understanding JSX for React

Start Terminal

Create react app

To create react app, there are two ways:

Use npm package manager

Use npx

With Npm, you have to install the package or app and keep it till further use. With NPX you can create the app and you don't have to keep it forever. IT saves hard disk space

Components Based Architecture

Think of a component as a small feature that makes up a piece of the user interface. If I were to describe a component within the scope of Facebook's UI:

- a chat window would be a component
- a comment feed would be another component
- a constantly updating friend list would represent yet another component.

These components exist within the same space, yet interact independently from one another

The need for components

- Components build off of the concept of AJAX requests, in which calls to the server are made directly from the client-side, allowing for the DOM to be dynamically updated without the need of a page refresh.

- Components each have their own interfaces that can make calls to the server and update their interfaces.
- Because components are independent, one component can refresh without affecting other components or the UI as a whole.

Dissecting the New App

WARNINGS – much like the side effects of a medicine

The folder structure looks complicated in the beginning but once you are familiar with it, it will be your favorite.

- All modules reside in node modules folder
- **.Gitignore** is used when working with github. Which files to push and which don't have to be pushed.
- **Package.json** contains all the packages used to be used in the app.
- **README** provides details of project in case you are pushing it to github.
- Public and **src** are two folders.
- All react components are based in **src**.

Understanding JSX

- Open a react app for development in Vscode
- Open the App.js file
- The Return function returns the html of the app
- Everything has to be in one tag

Working with JSX

- JSX is 99% Html rendered in JavaScript
- Only some syntax is different:
- className
- htmlFor
- JSX doesn't support two tags . The following is invalid
- Return (

```
<h1 > 123 </h1>  
<div> some content </div>
```
- The empty fragment `<> </>`
- If there's a scenario where you need to return two tags at the same time like in previous example, you should use an empty fragment
- The point is: JSX requires return of only one tag for rendering on page.

Set Emmet settings in VsCode

Before starting to write JSX, it's better to tell your compiler that the JavaScript in use must be specifically for React

Go to File > Preferences > Settings

Type in Emmet in search bar

Emmet: Include Languages

Add Item

Javascript: JavaScript react

Why Emmet settings?

This tells the vscode that if we are using .js extension, we expect react suggestions

Exploring JSX

Example 1:

Create a class name newClass

Color: #some color

Add class in app.css

In App.js, apply that class over a heading

Exploring JSX: adding variables

Example 2:

In App.js Before rendering, specify a variable e.g course = "Web 2"

In render function:

```
<h1> Hello class of {course} </h1>
```

Exploring JSX: ternary operators

Example 3:

```
const a = 5;
```

```
Var b = 10;
```

```
<p>{a > b ? "Greater": "Smaller"}</p>
```

Exploring JSX: working with functions

```
const greet = () => {  
  return <p>Hello</p>;  
};
```

```
function App() {  
  return (  
    <p>Function Call: {greet()}</p>  
  );  
}
```

Lab Tasks

1. Create a React app
2. Practice JSX concepts present in manual and create a lab manual
3. Create a simple Page in React. The page must include 3 pictures and a description of each picture in a paragraph.
4. Create a webpage to calculate and display your age in days using a js function.

LAB 5: Working with React JS Components

Objectives

1. To work with React Functional Components for a better understanding of Component based architecture.
2. Work with Bootstrap in React Projects and create react components from bootstrap.

Theoretical Description

Add Bootstrap to React JS

- Open the project
- Goto getbootstrap.com
- Get the cdn for js and stylesheet.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-ka7Sk0Gln4gmtz2MlQnikT1wXgYsOg+OMhuP+IlRH9sENBO0LRn5q+8nbTov4+1p" crossorigin="anonymous"></script>
```

- Paste them in index.html
- Bootstrap with React
- Add a component from bootstrap website
- Goto components > select Nav bar
- Copy code for Navbar
- Paste in App.js
- Check out closing tags and className
- Change the word NavBar with your project name
- We must have only two sections: Home and About
- So, remove excess tags from App.js

Props

- Props are arguments passed into React components.
- Props are passed to components via HTML attributes.
- props stand for properties.

Used to Pass Data

Props are also how you pass data from one component to another, as parameters. React Props are read-only! You will get an error if you try to change their value.

Understanding props and components

- Creating a custom component
- Create a folder “component”
- Component name must always be in capital Sentence case
- Create a component Navbar.js

Understanding props and components

- If you have installed ES7 React/... extension,
- Type in rfc and you will get React function-based components
- import React from 'react'

```
export default function Navbar() {  
  
  return (  
    <div>  
      This is navbar  
    </div>  
  )  
}
```

- Now open App.js and add this component in the App.js file.

```
<Navbar></Navbar>
```

- Observe that the following statement is already appended in the start of App.js

```
import Navbar from './Components/Navbar';
```

- Now the browser must display a message coming from Navbar component: This is navbar.
- This means component is being loaded.
- Now, the purpose of this component was to confine the Navbar huge code into a single component.
- So, we'll take all the code of navbar taken from bootstrap to the Navbar component
- Open **App.js**
- Copy all navbar related code and paste into **Navbar.js**
- App.js must contain one component: <Navbar/>
- Load the browser and the Navbar should load all fine.
- So, Now Navbar is a component that can be used in any website.
- However, we every Navbar can have a project name. So in case the component is used again, the title will have to be changed.
- Something like the following is favorable:
- <Navbar title = “MyProject” />
- This can customize the Navbar component every time it is used.
- <Navbar title ={prop} />
- This is the concept of a prop.

Understanding props

- Goto URL React.js > Docs > Main concepts > Components and Props
- Components let you split the UI into independent, reusable pieces, and think about each piece in isolation.
- Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called “props”) and return React elements describing what should appear on the screen.
- Coming back to the code
- Add a prop to the Navbar component
- `function Navbar(props) {`
- `// your code here`
- `}`

Find the following in the code:

```
<a className="navbar-brand" href="#">Navbar</a>  
now you can change it to
```

```
<a className="navbar-brand" href="#">{props.title}</a>
```

Now, changing the title in App.js will be reflected in the browser

In App.js

```
<Navbar title= “ChatApp” />  
<Navbar title= “CookingApp” />
```

Adding more props

Let’s change “about” Text in the navbar

Add a prop to it in Navbar component in the following text

```
<a className="nav-link" href="#">About</a>
```

To

```
<a className="nav-link" href="#">{props.aboutText}</a>
```

In the Navbar component add the following:

```
<Navbar title = “CodeApp” aboutText= “About Us” />
```

So, props are like html attributes of tags.

Lab Tasks

1. Follow the above steps to add bootstrap into your web page
2. Create a component from Bootstrap and Pass props into it.
3. Create your CV with three components from Bootstrap: Card, Accordion and Progress

LAB 6: Working with States in React JS

Objectives

To understand state objects and how they are rendered in React JS applications

Theoretical Description

The state is a built-in React object that is used to contain data or information about the component. A component's state can change over time; whenever it changes, the component re-renders. Create a Text analyzer by following the instructions:

Create a Form Control Component

- Add form from Bootstrap
- Rfc to create new component TextForm
- Add a container class in App.js
- Add TextForm into the container
- `import TextForm from './Components/TextForm';`
- Add a button to TextForm
- Heading prop
- Add heading with the help of props:

In App.js:

```
<TextForm heading="Enter Text"/>
```

- In TextForm.js:

Add props in default function:

```
export default function TextForm(props) {  
<h1>{props.heading}</h1>
```

Hooks – used in function based components

- `const [count, setCount] = useState(0);`
- Add to TextForm
- Replace to :
`const [text, setText] = useState('Enter Text here');`
- With the help of Hooks, we wish to update “text” variable wherever it is present in the page, without reloading

Always use it in function components

- Enter text here is the default value
- In TextForm.js, now set value to `{text}`

- Value = {text}
 - Try to Set text like normal variables:
 - Text = text2;
 - Gives error on webpage
 - State can't be set this way
 - Correct way is:
 - setText("updated text")
- ```
const [text, setText] = useState(0);

const handleOnChange = (event) => {
 setText(event.target.value);
}
```

## What is happening?

- Text is a state variable
- State is a plain JavaScript object used by React to represent an information about the component's current situation. It's managed in the component (just like any variable declared in a function). The difference is while a "normal" variable "disappears" when their function exits, the state variables are preserved by React.
- Now lets add logic to count words and characters

TextForm.js

```
return (
 <>
 <div className="container">
 <h1>{props.heading}</h1>
 <div className="mb-3">
 <textarea className="form-control" value={text} onChange={onChangeHandler} id="textBox" rows="3"></textarea>
 </div>
 </div>

 <div className="container my-2">
 </div>
 </>
)
```

- Add a summary
- <h1>Your page summary</h1>
- <p>123 words and 7890 characters</p>
- Change number of words

<h1>Your page summary</h1>

<p>{text.split(" ").length} words and {text.length} characters</p>

- Let's clear the text area
- Add a button in TextForm.js

```
<button className="btn btn-primary mx-2 my-2" onClick={onClearHandler}>Clear</button>
```

Define the onClearHandler:

```
const onClearHandler = () => {
 }
```

### How can you clear the text area?

```
setText("");
```

So our handler becomes

```
const onClearHandler = () => {
 setText("");
}
```

More functions can be found at any Text Analyzer online

### Asynchronous

- useState and setState both are asynchronous.
- They do not update the state immediately but have queues that are used to update the state object. This is done to improve the performance of the rendering of React components.

## Lab Tasks

1. Follow the above steps and
2. Convert all letters typed in the text area to UPPERCASE and LOWERCASE
3. Create a Web page and Enable Dark Mode and Light Mode with a toggle Button.

## LAB 7: React UI styling with States

### Objectives

1. Understanding JSX in React
2. Working with states
3. Loading dynamic CSS
4. Apply properties on buttons/input tags

### Theoretical Description

Dynamic CSS allows you to choose which stylesheet you be applied on screen. Follow the below instructions to load css dynamically.

#### Using useState with arrays

Create a component called CounterHook.js

Since we are Working with Arrays,

Default value will be empty array:

```
const [items, setItems] = useState([]);
export default function CounterHook() {
 const [items, setItems] = useState([]);
 return (
 <div>

 {items.map(item => (
 <li key={item.id}>{item.value}
))}

 </div>
)
}
```

#### Add a button to add items

```
<button onClick={addItem} className="primary">
 Add a number
</button>
```

#### Now add the event handler

```
const addItem = () => {
 setItems()
```

```
}
Push items into the setter function
setItems([])
```

```
setItems([...items, {
 id: items.length,
 value: "Item #" + items.length
}
])
```

What the code above does, is to spread over the items object and get all its properties, then overwrite the existing properties with the ones we're passing. Think of this spread thing as extracting all the individual properties one by one and transferring them to the new object. Now when you click the button

## Styling with React

Loading css dynamically

We can dynamically load styles in React JS

### Adding comments:

Use the braces

```
{/* */} - Multiline comments
{ //
} - single line comments.
```

### To dynamically load the style

Add a button to TextForm.js (or any component)

```
<button className="btn btn-primary">Enable Dark Mode
 let myStyle = {
 color: 'white',
 backgroundColor: 'grey'
 }
```

### This css will load dark mode

```
let myStyle = {
 color: 'white',
 backgroundColor : 'grey'
}
<div style={myStyle}></div>
```

### We'll now set the style using a button:

Declare the default style: Light mode

```

let myStyle = {
 color: 'black',
 backgroundColor : 'white'
}
const [modeStyle, setModeStyle] = useState('light')

const toggleStyle = () =>
{
 if (modeStyle === 'light')
 {

 setModeStyle('dark')
 document.body.style.backgroundColor = 'grey'
 document.body.style.color = 'white'
 }
 else
 {
 setModeStyle('light')
 document.body.style.backgroundColor = 'white'
 document.body.style.color = 'black'
 }
}

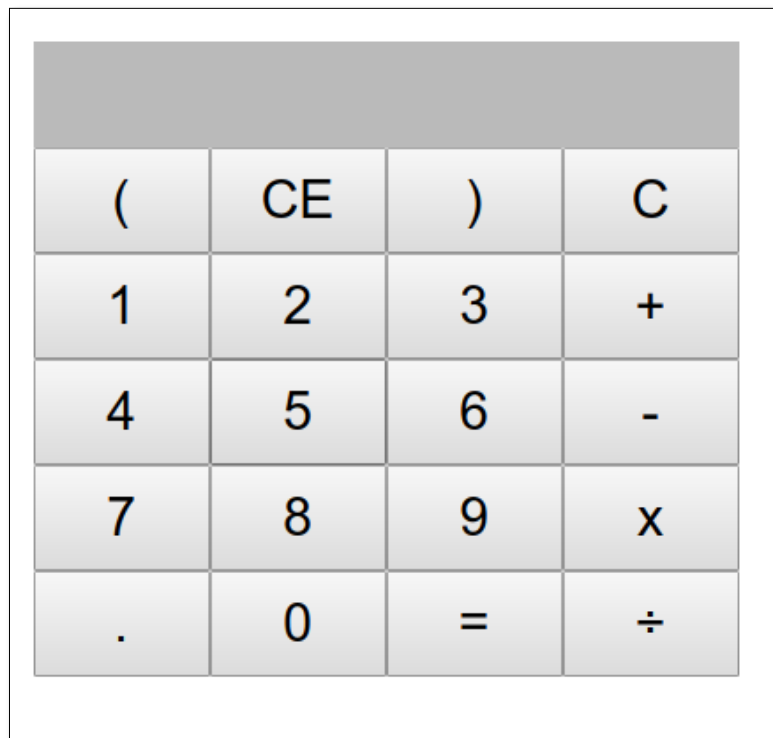
return (
 <>
 <div className="conatiner" mode = {modeStyle}>

 <Navbar title="Coding App" secondTab="About Us"/>
 <div className="container my-5">
 <TextForm title="Word Counter"/>

 <button className="btn btn-
primary" onClick={toggleStyle}>Change Mode</button>
 </div>
 </div>
 </>
);
}
export default App;

```

React JS allows for simple calculation which doesn't need databases to store values. Follow below steps to create a simple Calculator



First, we'll run the create-react-app command on our directory to create a basic react boilerplate application. Then we'll jump right on to building our components.

This calculator application will need two components. One is our Result component where we display the output, and the other is Keypad Component where we need to display our keypad.

In the `src` folder, we'll create a directory called *components*. This is where we will code our components. Go into that folder and create `ResultComponent.js`.

This is what the code would look like.

Our result component consists of just one `<p>` tag which displays content from props. Next, let's build a keypad component. Our keypad is just a bunch of buttons which do something when clicked. This is the code for our `KeypadComponent.js` file.

Since we need to send the click event to the parent (with information about which button was clicked), we will call *this.props.OnClick* function on every button click and *pass e.target.name* as an argument.

We'll define the logic of that function into our parent component, which is App.js.

In the App.js file, which is the parent of all our previous components and which renders all the main components, we will include all the child components first. We will also include "result" variable in *this.state* which is passed into our result component. This will allow us to manipulate our display.

Notice that I have passed *this.onClick* into Keypad component but I haven't defined it yet. Before defining that function, we need to do a few things. We need to create functions for our basic calculator features. In this calculator, I'll have three main features.

- *this.Calculate* => Calculate the result of our expression, this is triggered, when "=" button is pressed.
- *this.Reset* => Clear our output, this is triggered when "C" is pressed.
- *this.Backspace* => Clear the last character that was pressed. triggered on "CE".

Finally, we can create our *onClick* function and include it in our App.js file. The *onclick* function simple, reads the argument, which is name of the button clicked, and changes the state appropriately depending on the input that is passed. Finally, you can add a little bit of CSS to make it look nicer. With this we have completed our Calculator Application with React.JS.

## Lab Tasks

1. Implement Mode changing technique in your React from Light Mode to Dark Mode.
2. Implement a Calculator in React JS

## LAB 8: Working with React API fetch()

### Objectives

1. Understanding fetch() API in React
2. Working with useEffect Hook

### Theoretical Description

#### What is the Fetch API?

fetch() is a mechanism that lets you make simple AJAX (Asynchronous JavaScript and XML) calls with JavaScript.

Asynchronous means that you can use fetch to make a call to an external API without halting the execution of other instructions. That way, other functions on the site will continue to run even when an API call has not been resolved.

When a response (data) is sent back from the API, the asynchronous tasks (fetch) resume.

#### How to Use fetch() in JavaScript

When we talk about APIs, we also need to talk about endpoints. An endpoint is simply a unique URL you call to interact with another system.

Let's assume that we are making a request to an external API to get some data (like a blog post). For this, we'll use a simple GET request.

Simply call fetch() with the endpoint URL as the argument:

```
fetch('https://ubahthebuilder.tech/posts/1');
```

The response body for this endpoint will be information about a blog post:

```
{
 userId: 1,
 id: 1,
 title: 'A post by Kingsley',
 body: 'Brilliant post on fetch...',
};
```

#### Data response

The data returned from the API is not usually in a useable form. So, you'll need to convert the data to a form which your JavaScript can operate with. Thankfully, you can use the json() method to do just that:



Data response

```
fetch('https://ubahthebuilder.tech/posts/1')
.then(data => {
 return data.json();
})
.then(post => {
 console.log(post.title);
});
```

To make a simple GET request with fetch, you just need to pass in the URL endpoint as an argument.

## Data Fetch() in React using Axios

- Let's do the data fetch operation in React
- Create app
- you can use an existing app
- Install Axios
- MockAPI.io
- We'll use it to bring sample data to our app from another server.
- <https://mockapi.io/projects>
- mockAPI
- Create your account
- Create a resource
- Populate with data
- View data in URL

### Axios

**Axios** is a promise-based HTTP Client for node.js and the browser.

What it aims to improve: .fetch()

Fetch uses a two-step process when dealing with JSON data; after making an initial request you'll then need to call the .json() method on the response in order to receive the actual data object. If I were to have a fetch request to get back an object of relevant drivers for my carpool sourcing app, I might have something like the following:

```
const baseUrl = 'http://localhost:3000/api/v1'
fetch(`${baseUrl}/drivers`).then(res => res)
```

Let's see an example of API data fetch

Create students component

Import in App.js

Type in rfc for function based component

### API data fetch using Axios

Before we fetch data in React, first have a look at the data in the browser.

Example:

<https://61a484ad4c822c0017041cb2.mockapi.io/api/v1/StudentsApi/14>

If the content is loading in the form of json then you can proceed with it.

In Student.js, specify the url before the return statement

```
const url = 'https://61a484ad4c822c0017041cb2.mockapi.io/api/v1/StudentsApi/14';
```

Now we need to load users record. So, we will useState.

Paste below the url declaration

```
const [student, setStudent] = useState(null);
```

## Using Axios get method

```
axios.get(url)
.then(response => {
 console.log(response.data);
 setStudent(response.data)
})
```

Type this above the return statement

Now check response in browser console.

A number of repeated requests must be going on.

To cater that, we need to limit the requests.

For that we'll use useEffect hook

useEffect Hook

The useEffect Hook allows you to perform side effects in your components.

Some examples of side effects are: fetching data, directly updating the DOM, and timers.

useEffect accepts two arguments. The second argument is optional.

So, the useEffect hook makes sure the effect is only done once based on the changes in input (the second argument)

## Adding useEffect to the code

```
useEffect(() => {
 axios.get(url)
 .then(response => {
 console.log(response.data);
 setStudent(response.data)
 })
}, [url])
```

Make sure to import the useEffect Hook.

Display names and avatars

In the return function add the following code:

```
<div>
<h1>{student.name}</h1>
```

```

</div>
```

## Lab Tasks

1. Follow the above tutorial to fetch data from an API using mockapi.io
2. Create a Countdown timer with React Hooks using useEffect.

Hint: you can follow the following tutorial:

<https://www.digitalocean.com/community/tutorials/react-countdown-timer-react-hooks>

**LAB 9: Midterm Week**

## LAB 10: Working with React Routers

### Objectives

To enable students to work with Navigation using Routers in React JS applications.

### Theoretical Description

#### Routers in React: Types of Routers

React Router is a standard library for routing in React. It enables the navigation among views of various components in a React Application, allows changing the browser URL, and keeps the UI in sync with the URL.

#### The Router Component

- After installing react-router-dom, add its components to your React application.
- Adding React Router Components: The main Components of React Router are:
- BrowserRouter: BrowserRouter is a router implementation that uses the HTML5 history API(pushState, replaceState and the popstate event) to keep your UI in sync with the URL. It is the parent component that is used to store all of the other components.
- Routes: It's a new component introduced in the v6 and a upgrade of the component. The main advantages of Routes over Switch are:
- Routes are chosen based on the best match instead of being traversed in order.
- Route: Route is the conditionally shown component that renders some UI when its path matches the current URL.
- Link: Link component is used to create links to different routes and implement navigation around the application. It works like HTML anchor tag.

#### Types of Routers

On the basis of the part of URL that the router will use to track the content that the user is trying to view, React Router provides three different kinds of routers:

- Memory Router
- Browser Router
- Hash Router

#### Memory Router

Memory router keeps the URL changes in memory not in the user browsers. It keeps the history of the URL in memory (does not read or write to the address bar so the user can not use the browser's back button as well as the forward button. It doesn't change the URL in your browser. It is very useful for testing and non-browser environments like React Native.

Syntax: `import { MemoryRouter as Router } from 'react-router-dom';`

#### Browser Router

Browser Router: It uses HTML 5 history API (i.e. pushState, replaceState and popState API) to keep your UI in sync with the URL. It routes as a normal URL in the browser and assumes

that the server is handling all the request URL (eg., /, /about) and points to root index.html. It accepts forceRefresh props to support legacy browsers which doesn't support HTML 5 pushState API

```
import { BrowserRouter as Router } from 'react-router-dom';
```

## Hash Router:

- Hash router uses client-side hash routing. It uses the hash portion of the URL (i.e. window.location.hash) to keep your UI in sync with the URL.
- Hash portion of the URL won't be handled by the server, the server will always send the index.html for every request and ignore the hash value. It doesn't need any configuration in the server to handle routes. It is used to support legacy browsers which usually don't support HTML pushState API.
- It is very useful for the legacy browsers or you don't have a server logic to handle the client-side. This route isn't recommended to be used by the react-router-dom team.

```
import { HashRouter as Router } from 'react-router-dom';
```

Let us now try to understand the props associated with the Route component.

1. **exact**: It is used to match the exact value with the URL. For E.g., exact path='/about' will only render the component if it exactly matches the path but if we remove exact from the syntax, then UI will still be rendered even if the structure is like /about/10.
2. **path**: Path specifies a pathname we assign to our component.
3. **component**: It refers to the component which will render on matching the path.

Let's look at an example:

<https://www.geeksforgeeks.org/reactjs-router/>

## BrowserRouter

- Based on HTML5 history API
- Requires server handling dynamic requests (response to any possible URI)

## HashRouter

- Based on window.location.hash
- Can be used with static websites (server-less)

**MemoryRouter & StaticRouter** – mostly used for testing

react-router-dom

Npm install react-router-dom@5

<https://v5.reactrouter.com/web/guides/quick-start>

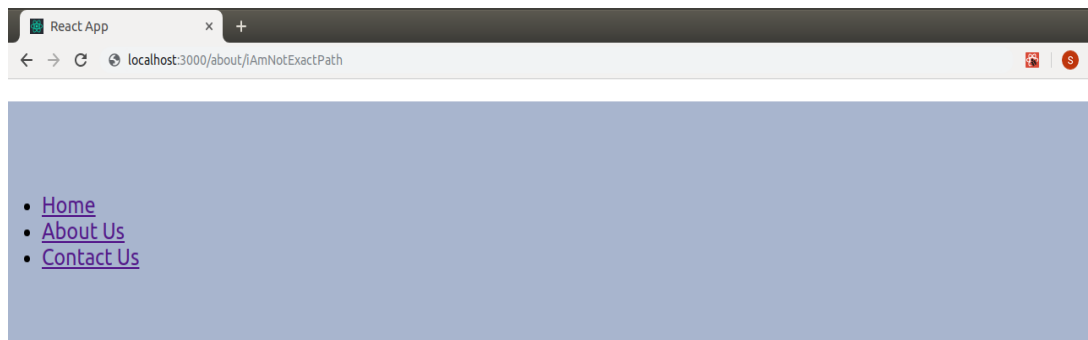
## Working with v6

- Switch is replaced by Routes
- In the path , you have to specify the Component

```
function App() {
 return (
 <Router>
 <Routes>
 <Route exact path="/" element={<Home />} />
 <Route path="/about" element={<About />} />
 <Route path="/Contact" element={<Contact />} />
 </Routes>
 </Router>
);
}
```

## Lab Task

Create the following in React JS using Routers. Use components for each view.



## LAB 11: Intro to MongoDB CRUD operations

### Objectives

1. Understanding MongoDB setup and command running

### Theoretical Description

CRUD operations *create*, *read*, *update*, and *delete* documents.

#### Create Operations

- Create or insert operations add new documents to a collection. If the collection does not exist, create operations also create the collection.
- You can insert a single document or multiple documents in a single operation.
- The MongoDB shell provides the following methods to insert documents into a collection:
- To insert a single document, use `db.collection.insertOne()`.
- To insert multiple documents, use `db.collection.insertMany()`.

The examples on this page reference the Atlas [sample dataset](#). You can create a free Atlas cluster and populate that cluster with sample data to follow along with these examples. To learn more, see [Get Started with Atlas](#).

#### Insert a Single Document

`db.collection.insertOne()` inserts a *single document* into a collection. If the document does not specify an `_id` field, MongoDB adds the `_id` field with an ObjectId value to the new document. See [Insert Behavior](#).

#### Example:

- To insert a new document into the `sample_mflix.movies` collection:

use `sample_mflix`

```
db.movies.insertOne(
{
 title: "The Favourite",
 genres: ["Drama", "History"],
 runtime: 121,
 rated: "R",
 year: 2018,
 directors: ["Yorgos Lanthimos"],
 cast: ["Olivia Colman", "Emma Stone", "Rachel Weisz"],
 type: "movie"
})
```

- `insertOne()` returns a document that includes the newly inserted document's `_id` field value.
- To retrieve the inserted document, [read the collection](#):



```
db.movies.find({ title: "The Favourite" })
```

- To ensure you return the document you inserted, you can instead query by `_id`.

## Insert Multiple Documents

- `db.collection.insertMany()` can insert *multiple documents* into a collection. Pass an array of documents to the method. If the documents do not specify an `_id` field, MongoDB adds the `_id` field with an ObjectId value to each document. See [Insert Behavior](#).

### Example:

- To insert two new documents into the `sample_mflix.movies` collection:
- use `sample_mflix`

```
db.movies.insertMany([
{
title: "Jurassic World: Fallen Kingdom",
genres: ["Action", "Sci-Fi"],
runtime: 130,
rated: "PG-13",
year: 2018,
directors: ["J. A. Bayona"],
cast: ["Chris Pratt", "Bryce Dallas Howard", "Rafe Spall"],
type: "movie"
},
{
title: "Tag",
genres: ["Comedy", "Action"],
runtime: 105,
rated: "R",
year: 2018,
directors: ["Jeff Tomsic"],
cast: ["Annabelle Wallis", "Jeremy Renner", "Jon Hamm"],
type: "movie"
}
])
```

- `insertMany()` returns a document that includes the newly inserted documents' `_id` field values.
- To [read documents in the collection](#):

```
db.movies.find({})
```

## Read Operations

- Read operations retrieve documents from a collection; i.e. query a collection for documents.
- You can specify criteria, or filters, that identify the documents to return.

- Use the `db.collection.find()` method in the MongoDB Shell to query documents in a collection.
- The examples on this page reference the Atlas [sample dataset](#). You can create a free Atlas cluster and populate that cluster with sample data to follow along with these examples. To learn more, see [Get Started with Atlas](#).

## Read All Documents in a Collection

To read all documents in the collection, pass an empty document as the query filter parameter to the find method. The query filter parameter determines the select criteria.

### Example:

To return all documents from the `sample_mflix.movies` collection:  
use `sample_mflix`

```
db.movies.find()
```

This operation is equivalent to the following SQL statement:

```
SELECT * FROM movies
```

## Specify Equality Condition

To select documents which match an equality condition, specify the condition as a `<field>:<value>` pair in the [query filter document](#).

### Example:

To return all movies where the `title` equals `Titanic` from the `sample_mflix.movies` collection:  
use `sample_mflix`

```
db.movies.find({ "title": "Titanic" })
```

This operation corresponds to the following SQL statement:

```
SELECT * FROM movies WHERE title = "Titanic"
```

## Specify Conditions Using Query Operators

Use [query operators](#) in a [query filter document](#) to perform more complex comparisons and evaluations. Query operators in a query filter document have the following form:

```
{ <field1>: { <operator1>: <value1> }, ... }
```

### Example:

To return all movies from the `sample_mflix.movies` collection which are either rated `PG` or `PG-13`:  
use `sample_mflix`

```
db.movies.find({ rated: { $in: ["PG", "PG-13"] } })
```

This operation corresponds to the following SQL statement:

```
SELECT * FROM movies WHERE rated in ("PG", "PG-13")
```

### Note:

Although you can express this query using the `$or` operator, use the `$in` operator rather than the `$or` operator when performing equality checks on the same field.

## Specify Logical Operators (`AND` / `OR`)

A compound query can specify conditions for more than one field in the collection's documents. Implicitly, a logical `AND` conjunction connects the clauses of a compound query so that the query selects the documents in the collection that match all the conditions.

### Example:

To return movies which were released in Mexico **and** have an IMDB rating of at least 7:

use sample\_mflix

```
db.movies.find({ countries: "Mexico", "imdb.rating": { $gte: 7 } })
```

Use the `$or` operator to specify a compound query that joins each clause with a logical `OR` conjunction so that the query selects the documents in the collection that match at least one condition.

### Example:

To return movies from the `sample_mflix.movies` collection which were released in 2010 **and** either won at least 5 awards or have a `genre` of `Drama`:

use sample\_mflix

```
db.movies.find({
 year: 2010,
 $or: [{ "awards.wins": { $gte: 5 } }, { genres: "Drama" }]
})
```

## Update Operations

- Update operations modify existing documents in a collection. You can update a single document or multiple documents in a single operation.
- You can specify criteria, or filters, that identify the documents to update. These filters use the same syntax as read operations.
- The MongoDB shell provides the following methods to update documents in a collection:
- To update a single document, use `db.collection.updateOne()`.
- To update multiple documents, use `db.collection.updateMany()`.
- To replace a document, use `db.collection.replaceOne()`.
- The examples on this page reference the Atlas [sample dataset](#). You can create a free Atlas cluster and populate that cluster with sample data to follow along with these examples. To learn more, see [Get Started with Atlas](#).

## Update Operator Syntax

- To update a document, MongoDB provides [update operators](#), such as `$set`, to modify field values.
- To use the update operators, pass to the update methods an update document of the form:

```
{
 <update operator>: { <field1>: <value1>, ... },
 <update operator>: { <field2>: <value2>, ... },
 ...
}
```

- Some update operators, such as `$set`, create the field if the field does not exist. See the individual [update operator](#) reference for details.

## Update a Single Document

Use the `db.collection.updateOne()` method to update the *first* document that matches a specified filter.

### Note:

MongoDB preserves a natural sort order for documents. This ordering is an internal implementation feature, and you should not rely on any particular structure within it. To learn more, see [natural order](#).

### Example:

To update the *first* document in the `sample_mflix.movies` collection where `title` equals `"Tag"`:  
use `sample_mflix`

```
db.movies.updateOne({ title: "Tag" },
{
 $set: {
 plot: "One month every year, five highly competitive friends
hit the ground running for a no-holds-barred game of tag"
 }
 { $currentDate: { lastUpdated: true } }
})
```

### The update operation:

- Uses the `$set` operator to update the value of the `plot` field for the movie `Tag`.
- Uses the `$currentDate` operator to update the value of the `lastUpdated` field to the current date. If `lastUpdated` field does not exist, `$currentDate` will create the field. See `$currentDate` for details.

## Update Multiple Documents

Use the `db.collection.updateMany()` to update all documents that match a specified filter.

### Example:

To update all documents in the `sample_airbnb.listingsAndReviews` collection to update where `security_deposit` is less than `100`:  
use `sample_airbnb`

```
db.listingsAndReviews.updateMany(
{ security_deposit: { $lt: 100 } },
```

```
{
 $set: { security_deposit: 100, minimum_nights: 1 }
}
```

The update operation uses the `$set` operator to update the value of the `security_deposit` field to `100` and the value of the `minimum_nights` field to `1`.

## Replace a Document

- To replace the entire content of a document except for the `_id` field, pass an entirely new document as the second argument to `db.collection.replaceOne()`.
- When replacing a document, the replacement document must contain only field/value pairs. Do not include [update operators](#) expressions.
- The replacement document can have different fields from the original document. In the replacement document, you can omit the `_id` field since the `_id` field is immutable; however, if you do include the `_id` field, it must have the same value as the current value.

## Example:

To replace the *first* document from the `sample_analytics.accounts` collection where `account_id: 371138`:

```
db.accounts.replaceOne(
 { account_id: 371138 },
 { account_id: 893421, limit: 5000, products: ["Investment", "Brokerage"] }
)
```

Run the following command to read your updated document:

```
db.accounts.findOne({ account_id: 893421 })
```

## Delete Operations

- Delete operations remove existing documents from a collection. You can remove a single document or multiple documents in a single operation.
- You can specify criteria, or filters, that identify the documents to remove. These filters use the same syntax as read operations.
- The MongoDB shell provides the following methods to delete documents from a collection:
- To delete multiple documents, use `db.collection.deleteMany()`.
- To delete a single document, use `db.collection.deleteOne()`.
- The examples on this page reference the Atlas [sample dataset](#). You can create a free Atlas cluster and populate that cluster with sample data to follow along with these examples. To learn more, see [Get Started with Atlas](#).

## Delete All Documents

To delete all documents from a collection, pass an empty [filter](#) document `{}` to the `db.collection.deleteMany()` method.

## Example:

To delete all documents from the `sample_mflix.movies` collection:  
use `sample_mflix`

```
db.movies.deleteMany({})
```

The method returns a document with the status of the operation. For more information and examples, see [deleteMany\(\)](#).

## Delete All Documents that Match a Condition

- You can specify criteria, or filters, that identify the documents to delete. The [filters](#) use the same syntax as read operations.
- To specify equality conditions, use `<field>:<value>` expressions in the query filter document.
- To delete all documents that match a deletion criteria, pass a filter parameter to the [deleteMany\(\)](#) method.

## Example:

To delete all documents from the `sample_mflix.movies` collection where the `title` equals "Titanic":  
use `sample_mflix`

```
db.movies.deleteMany({ title: "Titanic" })
```

The method returns a document with the status of the operation. For more information and examples, see [deleteMany\(\)](#).

## Delete Only One Document that Matches a Condition

To delete at most a single document that matches a specified filter (even though multiple documents may match the specified filter) use the [db.collection.deleteOne\(\)](#) method.

## Example:

To delete the *first* document from the `sample_mflix.movies` collection where the `cast` array contains "Brad Pitt":  
use `sample_mflix`

```
db.movies.deleteOne({ cast: "Brad Pitt" })
```

## Note:

MongoDB preserves a natural sort order for documents. This ordering is an internal implementation feature, and you should not rely on any particular structure within it. To learn more, see [natural order](#).

## Lab Tasks

1. Follow the above tutorial in Mongosh taken from :  
<https://www.mongodb.com/docs/mongodb-shell/crud/>

2. Create the following SQL table in MongoDB compass

ID	CITY	STATE	LAT_N	LONG_W
13	Phoenix	AZ	33	112
44	Denver	CO	40	105
66	Caribou	ME	47	68

3. Paste MongoDB Compass screenshot in manual.

## LAB 12: Intro to MERN stack App backend

### Objectives

1. Understanding MERN stack App backend

### Theoretical Description

MERN stands for MongoDB, Express, React, Node, after the four key technologies that make up the stack. MongoDB — document database. Express.js — Node.js web framework. React.js — a client-side JavaScript framework. To create a MERN backend, follow the following instructions:

#### Create db.js

- Create a separate file for db connection db.js
- Now connect to mongodb server
- `Const mongoose = require ('mongoose')`
- `Const mongoURI = "connection- string-here"`
- Remember we are coding in JS not JSX ( because this is not part of React Project but a part of Node.js server)
- Define connection and URI
- `const mongoose = require('mongoose')`
- `const mongoURI = "mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb"`
- `mongoose.connect(mongoURI, () => {`  
`})`
- The method takes two arguments, URI and a callback function
- Now you can now add some code within the call back function
- Add a log.  
`const connectToMongo = () => {`  
  
`mongoose.connect(mongoURI, () => {`  
 `console.log("Connected to Mongo Succesfully");`  
`})`  
`}`
- Now we'll export  
`module.exports = connectToMongo;`

So, this function will now export from here.

Connection is now a separate module

- Now go to index.js



```
const connectToMongo = require('./db')
connectToMongo();
```

- Meanwhile install nodemon as dependency
- `npm i -D nodemon`
- `-D` stands for Dependency
- You can also install globally
- Check installation with `nodemon -v`
- In terminal, type in `nodemon ./index.js`
- Our `index.js` is an express server
- Starting nodemon
- Now we'll run nodemon
- Add "serve": "nodemon index.js" in the package.json file

```
"scripts": {
 "test": "echo \"Error: no test specified\" && exit 1",
 "serve": "nodemon index.js"
},
```

Run command in terminal: `npm run serve`

- Express server
- Go to express server website > Getting started> HelloWorld
- URL:
- <https://expressjs.com/en/starter/hello-world.html>
- We'll take the basic HelloWorld code from here.
- To make `index.js` as an express server, take the following code:

```
const express = require('express')
const app = express()
const port = 3000
app.get('/', (req, res) => {
 res.send('Hello World!')
})
app.listen(port, () => {
 console.log(`Example app listening at http://localhost:${port}`)
})
```

Add express code to existing `index.js`:

## **Index.js**

```
const connectToMongo = require('./db')
const express = require('express')
connectToMongo();
const app = express()
const port = 3000
app.get('/', (req, res) => {
 res.send('Hello World!')
})
```

```
app.listen(port, () => {
 console.log(`Example app listening at http://localhost:${port}`)
})
```

## Lab Tasks

1. Follow the manual instructions
2. Create a MERN stack app backend on your machine.
3. Check difference between `app.get()` and `app.post()` with output.

## LAB 13: Working with MongoDB Schemas in MERN backend

### Objectives

To understand the MERN backend with Database Models and Schema.

### Theoretical Description

A schema is a JSON object that defines the the structure and contents of your data. You can use Realm's BSON schemas, which extend the [JSON Schema](#) standard, to define your application's data model and validate documents whenever they're created, changed, or deleted. Schemas represent *types* of data rather than specific values. Realm supports many built-in [schema types](#). These include primitives, like strings and numbers, as well as structural types, like objects and arrays, which you can combine to create schemas that represent custom *object types*.

For example, this is a basic schema for data about cars and some car objects that conform to the schema:

```
{
 "title": "car",
 "required": [
 "_id",
 "year",
 "make",
 "model",
 "miles"
],
 "properties": {
 "_id": { "bsonType": "objectId" },
 "year": { "bsonType": "string" },
 "make": { "bsonType": "string" },
 "model": { "bsonType": "string" },
 "miles": { "bsonType": "number" }
 }
}
```

Schemas are the specification for your application's data model. Once you've defined a schema, Realm provides you with additional tools and services to work with data that conforms to the schema.

- Now we'll see it in Thunderclient
- Go to Thunderclient under extensions. Click on New Request
- Under the GET request url, paste <http://localhost:3000>
- And click SEND

- The response is Hello World!
- It is coming from our Express Server
- Change some value in Hello world in index.js  
res.send('Hello Class of Web 2!')

Click Save and notice, “nodemon restarting due to changes” in the terminal  
Check Thunderclient request again, and observe how response is changed  
This response may change late, because res.get is asynchronous. Command  
This is one endpoint of our API

- Now let's create more endpoints:
- /api/v1/login
- /api/v1/SignUp
- MongoDB Schema:
- MongoDB Compass: Create Sample Database
- You can add collections and Documents.  
Documents are like JSON Objects, in general.
- Collections can have sub-collections too.
- Create a Database in MongoDB Compass and explore these features.
- Back to Code:
- Now, we'll create a schema for our database.
- Create a New Folder (within BackEnd) called Models.
- All mongoose models will go into this folder
- Create a Model named Notes.js (Notice the Capital N)
- Now create a schema for users

## **Users.js**

```
const mongoose = require('mongoose')
const userSchema = new Schema (
 {
 }
);
```

- Date should be with default method. (No need of parenthesis)
- Also set email to be unique: true
- Once Schema is defined, we'll export it via:  
module.exports = mongoose.model('user', userSchema)
- Now you must have two models under your model's folder

## **Lab Task**

Follow above manual and create Database Schemas for your project

## LAB 14: Understanding Backend Routers with MERN & API testing tools

### Objectives

- To enable backend Routers for better code optimization and handling URL patterns
- Understanding tools for API Testing

### Theoretical Description

Now we will work towards Routers.

#### What are Routes?

- A route is a section of Express code that associates an HTTP verb (GET, POST, PUT, DELETE, etc.), a URL path/pattern, and a function that is called to handle that pattern.
- In index.js add available routes (To specify GET requests)

#### // Available Routes

```
app.use('/api/auth' , require('./routes/auth'))
app.use('/api/notes' , require('./routes/notes'))
```

In the above code, we are linking app.use with another route file for better refactoring later on.

- Now create route files: notes.js and auth.js
- In auth.js

```
const express = require ('express');
const router = express.Router();
```

All notes related endpoints will be in Notes.js

All auth.js related endpoints will be in Auth.js

Now we'll use Router for get

#### Auth.js:

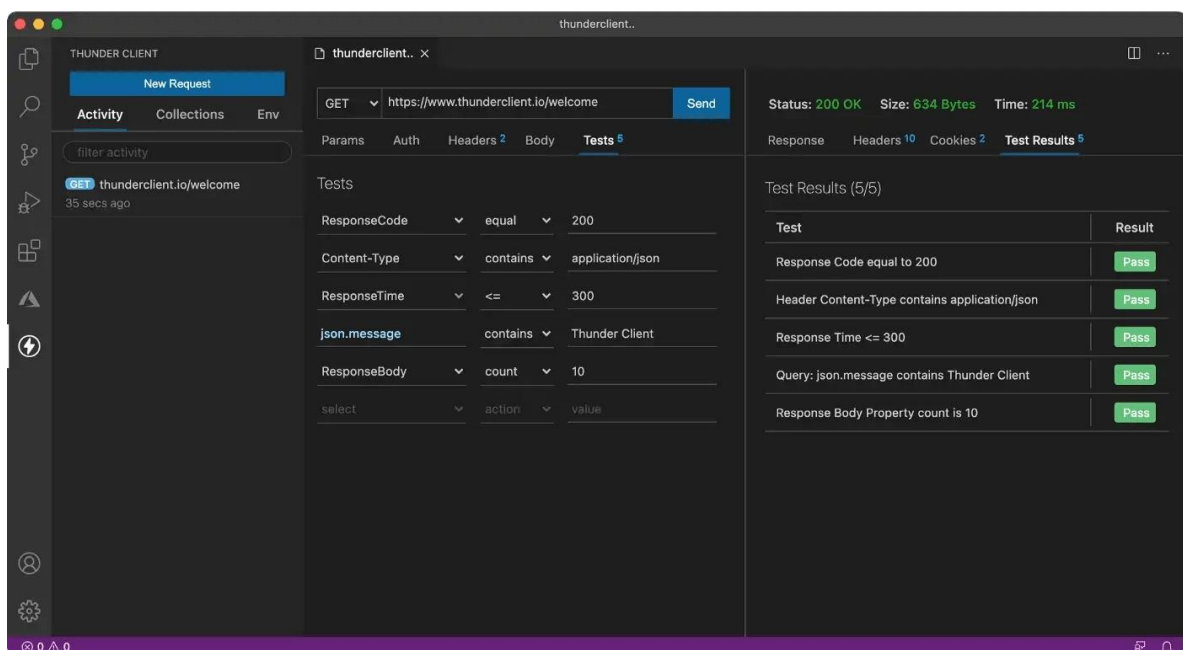
```
const express = require ('express');
const router = express.Router();
router.get('/', (re, res) => {
 obj={
 test: 'data',
 description:"This is sample obj"
 }
 res.json(obj)
})
```

- Now we have two endpoints
- Localhost:3000/api/auth

- Localhost:3000/api/notes
- Now we will add data to db
- For this, first we need to add specify json content under thunderclient
- Open the view with CTRL –
- Add json format as specified in the schema.
- Goto Headers > Content type> application/json
- Now we'll create a user

## API Testing

- One of the common requirements in web application development is testing your API's.
- In an era of API first apps, most of the functionalities in a web app are exposed as APIs to the outer world and to test these you need a tool.
- Postman has been widely used for this purpose.
- Postman has tons of features and often you may need it only to hit APIs with requests and see if you get the required response are not.
- You also may need additional features like sending headers and authentication credentials along with the request.
- Thunder Clients serves these purposes.



source: <https://rangav.medium.com/>

- It is better than VS Code for the below reasons:
- Postman consumes a lot of RAM and if your system has low configuration, postman may very well hang at times. Every additional hit you make to postman slows it down a bit.
- Thunder Client on the other hand is light weight.

2. It need not be installed separately

- Unlike postman Thunder Client comes as an extension to VS Code
- Alternatively for complex API development and testing, postman offers more features and can be preferred in place of Thunder Client.
- To install Thunder Client go to VS Code Extensions, search for Thunder Client and install.
- The tool appears on the left pane with the symbol of a thunder and you can start using it!

**Thunder Client:**

<https://www.thunderclient.com/>

**Postman:**

<https://www.postman.com/>

## Lab Tasks

1. Follow the above tutorial to create MERN stack BackEnd
2. Create the following SQL table in MongoDB compass

ID	CITY	STATE	LAT_N	LONG_W
13	Phoenix	AZ	33	112
44	Denver	CO	40	105
66	Caribou	ME	47	68

Paste MongoDB Compass screenshot in manual.

3. Create the API endpoints for READ, CREATE, UPDATE and DELETE
4. Create the backend for Login/Signup app in MERN stack (access the endpoints via url in browser or Thunderclient/POSTMAN) (You can use any web resource or tutorial)
5. Install Postman and Thunderclient
6. Follow the below tutorial for testing of thunderclient:  
<https://www.freecodecamp.org/news/thunder-client-for-vscode/>
7. Test the API endpoints for READ, CREATE, UPDATE and DELETE
8. Test the backend for Login/Signup app in MERN stack

## **LAB 15: Project Evaluation**



**LAB 16: Final Term Exam**

## Open Ended Lab (OEL)

**Course:** Web Engineering II

**Total Marks:** 15

**OEL Title:** Student Management Application using MERN Stack

**OEL Level:** 2

### **Objectives:**

- State the main purpose of this lab: To engage students in designing and developing a Student Management Application using the MERN stack, where they must determine their own approach to implementing required functionality.
- Identify what concepts or skills the student should learn or demonstrate by completing the lab:
  - Develop CRUD operations in a full-stack environment.
  - Design and connect front-end and back-end components.
  - Structure a MongoDB database to support application needs.
  - Apply creativity and problem-solving in designing features and workflows.
  - Build a functional, user-centered web application.

### **Background Information:**

The MERN stack (MongoDB, Express, React, Node.js) is a popular choice for building modern, full-stack web applications. It enables developers to create dynamic and scalable systems that combine a NoSQL database, a powerful back-end, and a reactive user interface.

Managing student information is a common requirement in educational institutions, where administrators need efficient tools to add, update, and retrieve records. Beyond basic operations, such systems often include features to improve usability, ensure data integrity, and provide actionable insights. This lab challenges students to build such a system, applying their technical knowledge while making design choices about the application's structure and features.

### **Problem:**

A university has requested a Student Management Application to support its administrative staff. The application must allow administrators to create, view, update, and delete student records. In addition, the university expects the application to provide functionality that makes record management more efficient and user-friendly.

You are responsible for designing and implementing this system using the MERN stack. While the basic requirement is CRUD functionality, you will need to decide how to structure the interface, what additional features will add value, and how to present data in a way that best serves the needs of the administrators. There is no single "correct" solution; different design decisions can meet the same requirements in unique ways.

## **Requirements:**

- Develop a React-based user interface that allows administrators to manage student records.
- Implement a RESTful back-end using Node.js and Express.
- Define and use a MongoDB schema to store student details (e.g., name, age, email, grade).
- Ensure that the application is functional, user-friendly, and supports efficient management of student data.
- Consider how to design the workflow, navigation, and additional supporting features in a way that enhances usability and meets the needs of administrators.